

Video Rental

CONTENTS

목차

Chapter1

Video Rental KATA 소개

Chapter2

Video Rental Refactoring 준비

Chapter3

단계 1. Unit Test

Chapter4

단계 2. Test Case Refactoring

Chapter5

단계 3. Legacy Code Refactoring

Chapter1

Video Rental KATA 소개

마틴파울러의 Refactroing 공식 예제

Video Rental Service

- 비디오 대여점에서
고객의 비디오 대여료 내역을 계산하고 출력하는 프로그램
- 고객이 대여한 비디오와 대여 기간을 표시
 - 비디오 종류와 대여 기간을 바탕으로 대여료를 계산

세 종류 장르의 비디오를 관리하는 비디오점

일반

유아

신작

비디오 렌탈

가격 계산 방법 (amount)

장르별, 대여 기간의 공식이 존재

포인트 적립 (frequentRentalPoint)

비디오 하나 빌릴 때 마다 Point 1점씩 준다.

특히, 신작을 이틀이상 빌리면 Point 1점을 더 준다.

일반
REGULAR

빌리면 Point 1점

유아
CHILDRENS

빌리면 Point 1점

신작
NEW_RELEASE

빌리면 Point 2점

고객이 비디오를 장바구니에 담는다.

- 고객이 원하는 비디오들을 담는다.
- 비디오 이름 / 장르 / 대여 희망일이 존재
영수증을 출력해주는 Legacy System.

"쥬스 아이스크림" (일반) - 3일 대여
"코로나 마스크" (일반) - 5일 대여
"토니스타크의 비밀" (신작) - 2일 대여
"홍부와 도깨비" (유아) - 1일 대여



Chapter2

Video Rental Refactoring 준비

실습 가이드

- 실습 카타 다운로드
 - <https://github.com/mincoding1/VideoRental>

Unit Test

1. 비디오 추가
2. statement() 호출시
3. 영수증 출력결과 비교

```
@Test
public void test() {
    Customer customer = new Customer("Bob");

    customer.addRental(new Rental(new Movie("Jaws", Movie.REGULAR), 2));
    customer.addRental(new Rental(new Movie("GoldenEye", Movie.REGULAR), 3));
    customer.addRental(new Rental(new Movie("ShortNew", Movie.NEW_RELEASE), 1));
    customer.addRental(new Rental(new Movie("LongNew", Movie.NEW_RELEASE), 2));
    customer.addRental(new Rental(new Movie("Bambi", Movie.CHILDRENS), 3));
    customer.addRental(new Rental(new Movie("Toy Story", Movie.CHILDRENS), 4));

    assertEquals("Rental Record for Bob\n" +
        "\tJaws\t2.0\n" +
        "\tGoldenEye\t3.5\n" +
        "\tShortNew\t3.0\n" +
        "\tLongNew\t6.0\n" +
        "\tBambi\t1.5\n" +
        "\tToy Story\t3.0\n" +
        "You owed 19.0\n" +
        "You earned 7 frequent renter points", customer.statement());
}
```

Movie / Rental

Movie

영화이름(title)과 장르 (priceCode)

Rental

Movie와 대여일(daysRented)

```
/**
 * The rental class represents a customer renting a movie.
 */
public class Rental {

    private Movie _movie;
    private int _daysRented;

    public Rental(Movie movie, int daysRented) {
        _movie = movie;
        _daysRented = daysRented;
    }

    public int getDaysRented() {
        return _daysRented;
    }

    public Movie getMovie() {
        return _movie;
    }
}
```

```
public class Movie {

    public static final int CHILDRENS = 2;
    public static final int NEW_RELEASE = 1;
    public static final int REGULAR = 0;

    private String _title;
    private int _priceCode;

    public Movie(String title, int priceCode) {
        _title = title;
        _priceCode = priceCode;
    }

    public int getPriceCode() {
        return _priceCode;
    }

    public void setPriceCode(int arg) {
        _priceCode = arg;
    }

    public String getTitle() {
        return _title;
    }
}
```

Customer.java

```
public String statement() {
    double totalAmount = 0;
    int frequentRenterPoints = 0;
    String result = "Rental Record for " + getName() + "\n";

    for (Rental each : _rentals) {
        double thisAmount = 0;

        //determine amounts for each line
        switch (each.getMovie().getPriceCode()) {
            case Movie.REGULAR:
                thisAmount += 2;
                if (each.getDaysRented() > 2)
                    thisAmount += (each.getDaysRented() - 2) * 1.5;
                break;
            case Movie.NEW_RELEASE:
                thisAmount += each.getDaysRented() * 3;
                break;
            case Movie.CHILDRENS:
                thisAmount += 1.5;
                if (each.getDaysRented() > 3)
                    thisAmount += (each.getDaysRented() - 3) * 1.5;
                break;
        }
    }
```

```
        // add frequent renter points
        frequentRenterPoints++;
        // add bonus for a two day new release rental
        if ((each.getMovie().getPriceCode() == Movie.NEW_RELEASE) && each.getDaysRented() > 1)
            frequentRenterPoints++;

        // show figures for this rental
        result += "\t" + each.getMovie().getTitle() + "\t" + String.valueOf(thisAmount) + "\n";
        totalAmount += thisAmount;
    }

    // add footer lines
    result += "Amount owed is " + String.valueOf(totalAmount) + "\n";
    result += "You earned " + String.valueOf(frequentRenterPoints) + " frequent renter points";

    return result;
}
```

Golden Mater Test

또는 Characterization test (캐릭터라이제이션 테스트)

유닛테스트가 없는 레거시 코드를 리팩토링할때 쓰임

실제 동작의 입력과 결과값을 저장해둔다.

많이 저장해둔다.

이걸로 유닛테스트 대용으로 사용한다.

리팩토링 하면서 지속적으로 테스트하면 된다.

- 주의점

가능한 모든 입력에 대한 출력을 Characterization Test로 생성하는 것을 불가능하다.

테스트 작성자는 자신의 판단으로 테스트가 얼마나 적합한지 결정할 수 있어야 한다.



Chapter3

1. Unit Test

Golden Master Test (Characterization Test)

Hamcrest 란?

✓가독성을 위한 JUnit 기반 Test Library

- AssertEqual 대신, AssertThat을 사용하여 가독성을 높이기 위해 만들어졌다.

✓다양한 Matcher 지원

- Pass 인지 아닌지,
조건을 검사하는 여러가지 기능들을 추가적으로 사용 가능

```
@Test
public void getSum() {
    Cal c = new Cal();
    int ret = c.getSum( a: 10, b: 20);

    assertEquals(ret, actual: 30);
}
```

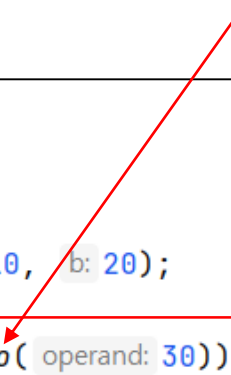
JUnit

```
@Test
public void getSum() {
    Cal c = new Cal();
    int ret = c.getSum( a: 10, b: 20);

    assertThat(ret, equalTo(operand: 30));
}
```

Hamcrest

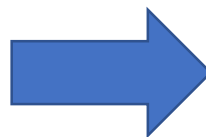
Matcher



Hamcrest 설치하기

✓Maven Repository 에서 검색 후, jar 파일 추가.

The screenshot shows the Maven Repository search results for 'hamcrest'. The search bar at the top contains 'hamcrest'. Below the search bar, there is a section for 'Indexed Artifacts (33.4M)' with a line graph showing the number of projects over time. To the right, the 'Group: Hamcrest' is listed, and the 'Sort' is set to 'popular'. The first result is '1. Hamcrest All' by 'org.hamcrest', which is highlighted with a red box. Below the result, it says 'A self-contained hamcrest jar con' and 'Last Release on Jul 9, 2012'.



The screenshot shows the details for the 'Hamcrest All 1.3' artifact. The title is 'Hamcrest All » 1.3'. Below the title, it says 'A self-contained hamcrest jar containing all of the sub'. The 'License' is 'BSD'. The 'Categories' are 'Testing Frameworks & Tools'. The 'Tags' are 'matching', 'hamcrest', and 'testing'. The 'Date' is 'Jul 09, 2012'. The 'Files' section shows 'pom (650 bytes)' and 'jar (299 KB)', with the 'jar' file highlighted by a red box. The 'Repositories' section lists 'Central', 'Redhat GA', 'Sonatype', and 'Sovrin'. The 'Ranking' section shows '#63 in MvnRepository (See Top Artif)' and '#8 in Testing Frameworks & Tools'. The 'Used By' section shows '7,431 artifacts'.

이번 실습에서는 Hamcrest을 체험 위주로 다룬다.
차후에는 Hamcrest 대신, Spring에서 쓰이는 AssertJ 심도 있게 다룰 예정

[참고] Hamcrest 사용을 위한 import 추가.

- ✓ hamcrest Library 설치 후,
수동으로 아래 두 static import문을 추가해주면 된다.

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

import static org.hamcrest.MatcherAssert.*;
import static org.hamcrest.CoreMatchers.*;

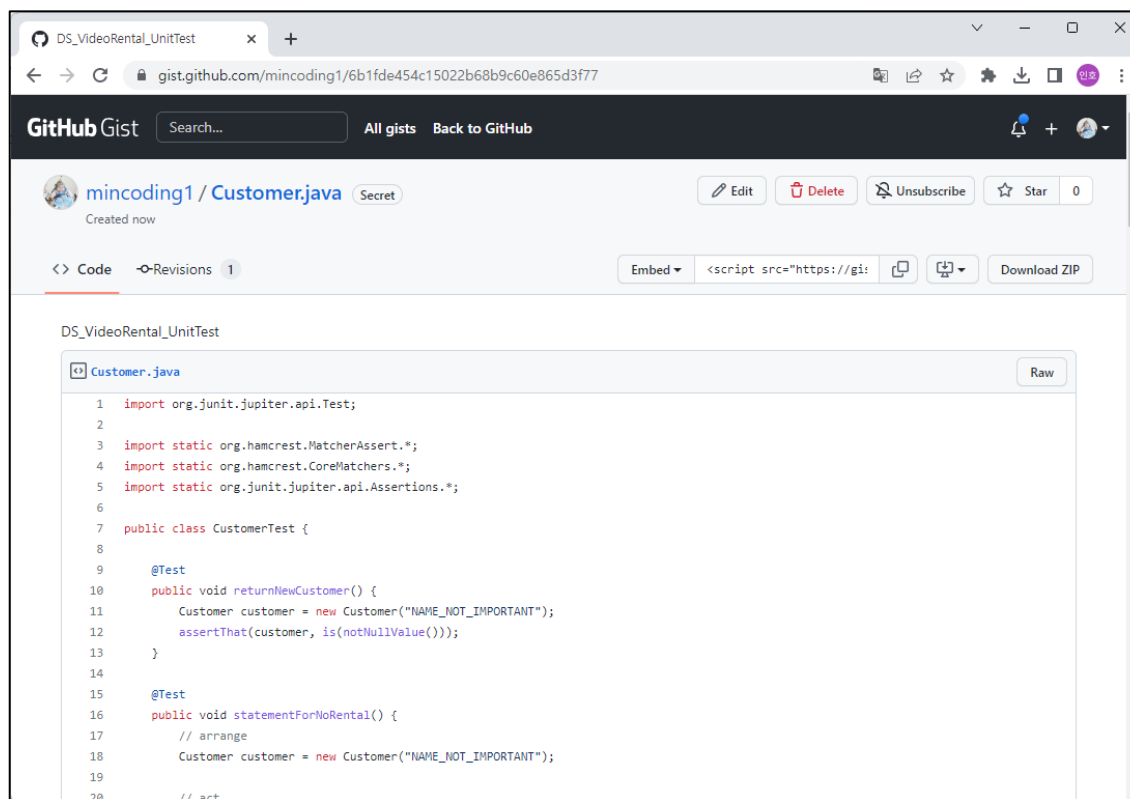
class MainTest {

    @Test
    void test() {
        assertThat(actual: 1 + 1, is(value: 2));
    }
}
```

완성된 유닛테스트 코드

✓유닛테스트 실습 1 ~ 8 까지 소스코드

- <https://gist.github.com/mincoding1/6b1fde454c15022b68b9c60e865d3f77>



The screenshot shows a web browser displaying a GitHub Gist page. The URL in the address bar is <https://gist.github.com/mincoding1/6b1fde454c15022b68b9c60e865d3f77>. The page header shows the GitHub logo and the text "mincoding1 / Customer.java". Below the header, there are buttons for "Edit", "Delete", "Unsubscribe", "Star", and "0". The main content area shows the code for "Customer.java" with line numbers 1 through 20. The code is a Java unit test for the "Customer" class, using JUnit 4 and Hamcrest. It includes imports for JUnit and Hamcrest, and defines two test methods: "returnNewCustomer()" and "statementForNoRental()".

```
1 import org.junit.jupiter.api.Test;
2
3 import static org.hamcrest.MatcherAssert.*;
4 import static org.hamcrest.CoreMatchers.*;
5 import static org.junit.jupiter.api.Assertions.*;
6
7 public class CustomerTest {
8
9     @Test
10     public void returnNewCustomer() {
11         Customer customer = new Customer("NAME_NOT_IMPORTANT");
12         assertThat(customer, is(notNullValue()));
13     }
14
15     @Test
16     public void statementForNoRental() {
17         // arrange
18         Customer customer = new Customer("NAME_NOT_IMPORTANT");
19
20         // act
```

[실습] Test Case

✓1. Customer 생성 테스트

- Customer 객체를 생성하는 테스트를 작성한다.

CustomerTest.java

```
@Test
public void returnNewCustomer() {

    Customer customer = new Customer("NAME_NOT_IMPORTANT");

    assertThat(customer, is(notNullValue()));
}
```

[실습] Test Case

✓2. Movie를 Rental 하지 않은 경우 테스트

- 비디오를 대여하지 않은 경우의 statement 출력 결과 비교하는 테스트를 작성한다.

CustomerTest.java

```
@Test
public void statementForNoRental() {

    // arrange
    Customer customer = new Customer("NAME_NOT_IMPORTANT");

    // act
    String statement = customer.statement();

    // assert
    assertThat(statement, is("Rental Record for NAME_NOT_IMPORTANT\n"
        + "Amount owed is 0.0\n"
        + "You earned 0 frequent renter pointers"));
}
```

[실습] Test Case

✓ 3. Amount 계산 로직 테스트 : Regular movie

- if 조건 분기 경계값으로 테스트

CustomerTest.java

```
@Test
public void statementForRegularMovieRentalForLessThan3Days() {

    // arrange
    Customer customer = new Customer("NAME_NOT_IMPORTANT");
    Movie movie = new Movie("TITLE_NOT_IMPORTANT", Movie.REGULAR);
    int daysRented = 2;
    Rental rental = new Rental(movie, daysRented);
    customer.addRental(rental);

    // act
    String statement = customer.statement();

    // assert
    assertThat(statement, is("Rental Record for NAME_NOT_IMPORTANT\n"
        + "\t2.0(TITLE_NOT_IMPORTANT)\n"
        + "Amount owed is 2.0\n"
        + "You earned 1 frequent renter pointers"));
}
```

[실습] Test Case

✓4. Amount 계산 로직 테스트 : New release movie

CustomerTest.java

```
@Test
public void statementForNewReleaseMovie() {

    // arrange
    Customer customer = new Customer("NAME_NOT_IMPORTANT");
    Movie movie = new Movie("TITLE_NOT_IMPORTANT", Movie.NEW_RELEASE);
    int daysRented = 1;
    Rental rental = new Rental(movie, daysRented);
    customer.addRental(rental);

    // act
    String statement = customer.statement();

    // assert
    assertThat(statement, is("Rental Record for NAME_NOT_IMPORTANT\n"
        + "\t3.0(TITLE_NOT_IMPORTANT)\n"
        + "Amount owed is 3.0\n"
        + "You earned 1 frequent renter pointers"));
}
```

[실습] Test Case

✓5. Amount 계산 로직 테스트 : Childrens movie (1/2)

- if 조건 분기 경계값으로 테스트

CustomerTest.java

```
@Test
public void statementForChildrensMovieRentalMoreThan3Days() {

    // arrange
    Customer customer = new Customer("NAME_NOT_IMPORTANT");
    Movie movie = new Movie("TITLE_NOT_IMPORTANT", Movie.CHILDRENS);
    int daysRented = 4;
    Rental rental = new Rental(movie, daysRented);
    customer.addRental(rental);

    // act
    String statement = customer.statement();

    // assert
    assertThat(statement, is("Rental Record for NAME_NOT_IMPORTANT\n"
        + "\t3.0(TITLE_NOT_IMPORTANT)\n"
        + "Amount owed is 3.0\n"
        + "You earned 1 frequent renter pointers"));
}
```

[실습] Test Case

✓ 6. Amount 계산 로직 테스트 : Childrens movie (2/2)

- if 조건 분기 경계값으로 테스트

CustomerTest.java

```
@Test
public void statementForChildrensMovieRentalLessThan4Days() {

    // arrange
    Customer customer = new Customer("NAME_NOT_IMPORTANT");
    Movie movie = new Movie("TITLE_NOT_IMPORTANT", Movie.CHILDRENS);
    int daysRented = 3;
    Rental rental = new Rental(movie, daysRented);
    customer.addRental(rental);

    // act
    String statement = customer.statement();

    // assert
    assertThat(statement, is("Rental Record for NAME_NOT_IMPORTANT\n"
        + "\t1.5(TITLE_NOT_IMPORTANT)\n"
        + "Amount owed is 1.5\n"
        + "You earned 1 frequent renter pointers"));
}
```


[실습] Test Case

✓7. Frequent renter points 계산 로직

- 보너스 포인트 로직은 New Release Movie에만 해당

CustomerTest.java

```
@Test
public void statementForNewReleaseMovieRentalMoreThan1Day() {

    // arrange
    Customer customer = new Customer("NAME_NOT_IMPORTANT");
    Movie movie = new Movie("TITLE_NOT_IMPORTANT", Movie.NEW_RELEASE);
    int daysRented = 2;
    Rental rental = new Rental(movie, daysRented);
    customer.addRental(rental);

    // act
    String statement = customer.statement();

    // assert
    assertThat(statement, is("Rental Record for NAME_NOT_IMPORTANT\n"
        + "\t6.0(TITLE_NOT_IMPORTANT)\n"
        + "Amount owed is 6.0\n"
        + "You earned 2 frequent renter pointers"));
}
```

[실습] Test Case

✓8. Movie 여러 개를 대여한 경우

CustomerTest.java

```
@Test
public void statementForFewMovieRental() {

    Customer customer = new Customer("NAME_NOT_IMPORTANT");
    Movie regularMovie = new Movie("TITLE_NOT_IMPORTANT", Movie.REGULAR);
    Movie newReleaseMovie = new Movie("TITLE_NOT_IMPORTANT", Movie.NEW_RELEASE);
    Movie childrensMovie = new Movie("TITLE_NOT_IMPORTANT", Movie.CHILDRENS);
    customer.addRental(new Rental(regularMovie, 1));
    customer.addRental(new Rental(newReleaseMovie, 4));
    customer.addRental(new Rental(childrensMovie, 4));

    String statement = customer.statement();

    assertThat(statement, is("Rental Record for NAME_NOT_IMPORTANT\n"
        + "\t2.0(TITLE_NOT_IMPORTANT)\n"
        + "\t12.0(TITLE_NOT_IMPORTANT)\n"
        + "\t3.0(TITLE_NOT_IMPORTANT)\n"
        + "Amount owed is 17.0\n"
        + "You earned 4 frequent renter pointers"));
}
```

Chapter4

2. Test Case Refactoring

[실습] Test Case Refactoring

✓1. Duplicated Code

- 중복해서 선언된 지역변수를 필드로 변환하여 중복을 제거한다.
- 모든 유닛테스트(9개)에 존재하는 customer 를 전부 제거한다.

CustomerTest.java

```
@Test
public void returnNewCustomer() {
    Customer customer = new Customer("NAME_NOT_IMPORTANT");
    assertThat(customer, is(notNullValue()));
}

@Test
public void statmentForNoRental() {
    // arrange
    Customer customer = new Customer("NAME_NOT_IMPORTANT");

    // act
    String statement = customer.statement();

    // assert
    assertThat(statement, is("Rental Record for NAME_NOT_IMPORTANT\n"
        + "Amount owed is 0.0\n"
        + "You earned 0 frequent renter pointers"));
}
```



CustomerTest.java

```
Customer customer = new Customer("NAME_NOT_IMPORTANT");

@Test
public void returnNewCustomer() {
    assertThat(customer, is(notNullValue()));
}

@Test
public void statmentForNoRental() {
    // act
    String statement = customer.statement();

    // assert
    assertThat(statement, is("Rental Record for
NAME_NOT_IMPORTANT\n"
        + "Amount owed is 0.0\n"
        + "You earned 0 frequent renter pointers"));
}
```

지역변수 customer를 모두 전역으로 변경 (총 9 곳 변경)

[실습] Test Case Refactoring

✓2. Replace Magic Number with Symbolic Constant

- 의미를 알 수 없는 숫자, 문자열은 의미를 알아볼 수 있는 상수로 치환한다.
- IDE의 Refactor 기능 (**Introduce Constant**) 을 쓰면 편리하게 변경 가능

CustomerTest.java

```
Customer customer = new Customer("NAME_NOT_IMPORTANT");
```



CustomerTest.java

```
public static final String NAME = "NAME_NOT_IMPORTANT";  
Customer customer = new Customer(NAME);
```

[실습] Test Case Refactoring

✓2. Replace Magic Number with Symbolic Constant

- 의미를 알 수 없는 숫자, 문자열은 의미를 알아볼 수 있는 상수로 치환한다.
 - 모든 "TITLE_NOT_IMPORTANT" 문자열을 TITLE 상수로 치환한다.
- IDE의 Refactor 기능 (Introduce Constant) 을 쓰면 편리하게 변경 가능

CustomerTest.java

```
Movie movie = new Movie("TITLE_NOT_IMPORTANT", Movie.REGULAR);
```



CustomerTest.java

```
public static final String TITLE = "TITLE_NOT_IMPORTANT";
```

```
@Test
```

```
public void statementForRegularMovieRentalForLessThan3Days() {
```

```
    // arrange
```

```
    Movie movie = new Movie(TITLE, Movie.REGULAR);
```

```
    int daysRented = 2;
```

```
    Rental rental = new Rental(movie, daysRented);
```

```
    customer.addRental(rental);
```

```
    // act
```

```
    String statement = customer.statement();
```

[실습] Test Case Refactoring

✓3. Inline Temp

- 간단한 수식을 대입받는 임시변수를 수식으로 변환한다.
- Refactor 기능 (Inline Variable)을 사용하면 편리하게 변환이 가능하다.
 - 모든 statement 변수 (총 8개) Inline Variable 진행

CustomerTest.java

```
@Test
public void statmentForNoRental() {
    // act
    String statement = customer.statment();

    // assert
    assertThat(statement, is("Rental Record for
NAME_NOT_IMPORTANT\n"
        + "Amount owed is 0.0\n"
        + "You earned 0 frequent renter pointers"));
}
```

CustomerTest.java

```
@Test
public void statmentForNoRental() {
    // assert
    assertThat(customer.statment(), is("Rental Record for NAME_NOT_IMPORTANT\n"
        + "Amount owed is 0.0\n"
        + "You earned 0 frequent renter pointers"));
}
```

[실습] Test Case Refactoring

✓4-1. Long Method → extract method 를 위한 준비작업

- 상수를 변수로 만들기 : Refactor 기능 (Introduce Variable 하기)
- 소스코드 순서 변경

CustomerTest.java

```
@Test
public void statementForRegularMovieRentalForLessThan3Days() {

    // arrange
    Movie movie = new Movie(TITLE, Movie.REGULAR);
    int daysRented = 2;
    Rental rental = new Rental(movie, daysRented);
    customer.addRental(rental);

    ..
}
```



CustomerTest.java

```
@Test
public void statementForRegularMovieRentalForLessThan3Days() {

    // arrange
    int daysRented = 2;
    int priceCode = Movie.REGULAR;
    Movie movie = new Movie(TITLE, priceCode);
    Rental rental = new Rental(movie, daysRented);
    customer.addRental(rental);

    ...
}
```


[실습] Test Case Refactoring

✓4-2. Long Method → extract method

▪ Refactor 기능 (extract method)

CustomerTest.java

```
@Test
public void statementForRegularMovieRentalForLessThan3Days() {

    // arrange
    int daysRented = 2;
    int priceCode = Movie.REGULAR;
    Movie movie = new Movie(TITLE, priceCode);
    Rental rental = new Rental(movie, daysRented);
    customer.addRental(rental);

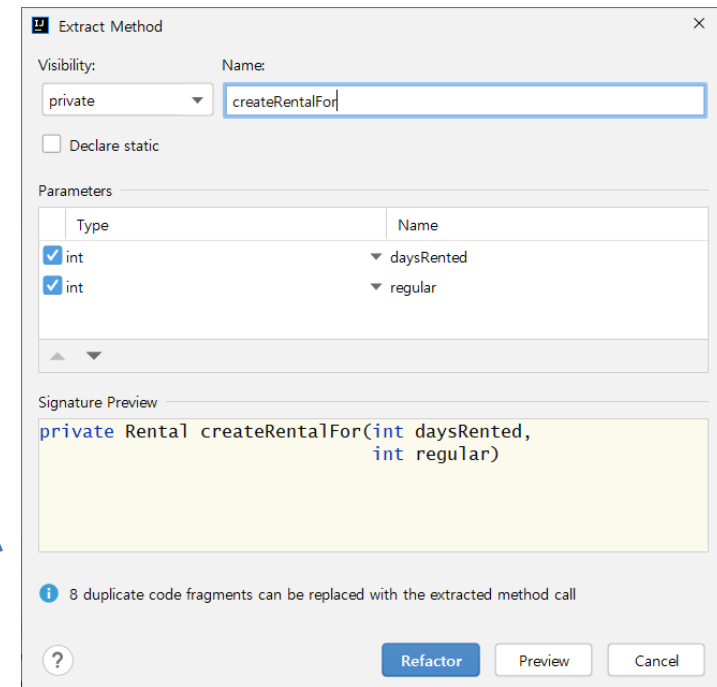
    ...
}
```

CustomerTest.java

```
@Test
public void statementForRegularMovieRentalForLessThan3Days() {

    // arrange
    int daysRented = 2;
    int priceCode = Movie.REGULAR;
    Rental rental = createRentalFor(daysRented, priceCode);
    customer.addRental(rental);

    // assert
    assertThat(customer.statement(), is("Rental Record for NAME_NOT_IMPORTANT\n"
        + "\t2.0(TITLE_NOT_IMPORTANT)\n"
        + "Amount owed is 2.0\n"
        + "You earned 1 frequent renter pointers"));
}
```



[실습] Test Case Refactoring

✓4-3. Long Method 정리

- Refactor 기능 (Inline Variable)

CustomerTest.java

```
@Test
public void statementForRegularMovieRentalForLessThan3Days() {

    // arrange
    int daysRented = 2;
    int priceCode = Movie.REGULAR;
    Rental rental = createRentalFor(daysRented, priceCode);
    customer.addRental(rental);

    // assert
    assertThat(customer.statement(), is("Rental Record for NAME_NOT_IMPORTANT\n"
        + "\t2.0(TITLE_NOT_IMPORTANT)\n"
        + "Amount owed is 2.0\n"
        + "You earned 1 frequent renter pointers"));
}
```



CustomerTest.java

```
@Test
public void statementForRegularMovieRentalForLessThan3Days() {

    // arrange
    customer.addRental(createRentalFor(2, Movie.REGULAR));

    // assert
    assertThat(customer.statement(), is("Rental Record for NAME_NOT_IMPORTANT\n"
        + "\t2.0(TITLE_NOT_IMPORTANT)\n"
        + "Amount owed is 2.0\n"
        + "You earned 1 frequent renter pointers"));
}
```

[실습] Test Case Refactoring

✓4-4. 모든 Unit Test 함수에, 제작한 Method를 사용하도록 코드를 변경한다.

- 나머지 6개 Unit Test 함수에, 수동으로 변경하기

CustomerTest.java

```
@Test
public void statementForRegularMovieRentalForMoreThan2Days() {
    // arrange
    Movie movie = new Movie(TITLE, Movie.REGULAR);
    int daysRented = 3;
    Rental rental = new Rental(movie, daysRented);
    customer.addRental(rental);

    // assert
    assertThat(customer.statement(), is( value: "Rental Record for NAME_NOT_FOUND
        + "\t3.5(TITLE_NOT_IMPORTANT)\n"
        + "Amount owed is 3.5\n"
        + "You earned 1 frequent renter pointers")));
}
```



CustomerTest.java

```
@Test
public void statementForRegularMovieRentalForMoreThan2Days() {
    // arrange
    customer.addRental(createRentalFor( daysRented: 3, Movie.REGULAR));

    // assert
    assertThat(customer.statement(), is( value: "Rental Record for NAME_NOT_FOUND
        + "\t3.5(TITLE_NOT_IMPORTANT)\n"
        + "Amount owed is 3.5\n"
        + "You earned 1 frequent renter pointers")));
}
```

[미션] Code Coverage 모두 100% 만들기

- ✓현재 Code Coverage를 측정한다.
- ✓부족한 Coverage를 채울 수 있도록 Unit Test 하나를 더 추가한다.

Coverage: CustomerTest ×			
↑ 100% classes, 95% lines covered in package 'com.videorental'			
Element	Class, %	Method, %	Line, %
Customer	100% (1/1)	100% (4/4)	100% (30/30)
Movie	100% (1/1)	75% (3/4)	75% (6/8)
Rental	100% (1/1)	100% (3/3)	100% (6/6)

3. Legacy Code Refactoring

실습 도중에, Unit Test의 Code Coverage가 줄어들면
다시 100%로 채워야 하지만, 이 부분은 생략하자.

[실습] Legacy Code Refactoring

✓1. Switch문은 하나의 메서드로 빼는 것이 좋다.

- Step 1. 변수(thisAmount)를 사용 직전에 선언하도록 한다.

Customer.java

```
public String statement() {
    double totalAmount = 0;
    int frequentRenterPoints = 0;
    Iterator<Rental> iterator = rentals.iterator();
    String result = "Rental Record for " + getName() + "\n";

    while ( iterator.hasNext() ) {
        double thisAmount = 0;
        Rental each = (Rental) iterator.next();
        // determine amounts for each line

        switch (each.getMovie().getPriceCode()) {
            case Movie.REGULAR:
                thisAmount += 2;
                if (each.getDaysRented() > 2)
                    thisAmount += (each.getDaysRented() - 2) * 1.5;
                break;

            case Movie.NEW_RELEASE:
                thisAmount += each.getDaysRented() * 3;
                break;

            case Movie.CHILDRENS:
                thisAmount += 1.5;
                if (each.getDaysRented() > 3)
                    thisAmount += (each.getDaysRented() - 3) * 1.5;
                break;
        }
    }
```



Customer.java

```
public String statement() {
    double totalAmount = 0;
    int frequentRenterPoints = 0;
    Iterator<Rental> iterator = rentals.iterator();
    String result = "Rental Record for " + getName() + "\n";

    while ( iterator.hasNext() ) {
        Rental each = (Rental) iterator.next();
        // determine amounts for each line

        double thisAmount = 0;
        switch (each.getMovie().getPriceCode()) {
            case Movie.REGULAR:
                thisAmount += 2;
                if (each.getDaysRented() > 2)
                    thisAmount += (each.getDaysRented() - 2) * 1.5;
                break;

            case Movie.NEW_RELEASE:
                thisAmount += each.getDaysRented() * 3;
                break;

            case Movie.CHILDRENS:
                thisAmount += 1.5;
                if (each.getDaysRented() > 3)
                    thisAmount += (each.getDaysRented() - 3) * 1.5;
                break;
        }
    }
```

[실습] Legacy Code Refactoring

✓ 1. Switch문은 하나의 메서드로 빼는 것이 좋다.

- step 2. extract method

Customer.java

```
public String statement() {
    double totalAmount = 0;
    int frequentRenterPoints = 0;
    Iterator<Rental> iterator = rentals.iterator();
    String result = "Rental Record for " + getName() + "\n";

    while ( iterator.hasNext() ) {
        Rental each = (Rental) iterator.next();
        // determine amounts for each line

        double thisAmount = 0;
        switch (each.getMovie().getPriceCode()) {
            case Movie.REGULAR:
                thisAmount += 2;
                if (each.getDaysRented() > 2)
                    thisAmount += (each.getDaysRented() - 2) * 1.5;
                break;

            case Movie.NEW_RELEASE:
                thisAmount += each.getDaysRented() * 3;
                break;

            case Movie.CHILDRENS:
                thisAmount += 1.5;
                if (each.getDaysRented() > 3)
                    thisAmount += (each.getDaysRented() - 3) * 1.5;
                break;
        }
    }
}
```



Customer.java

```
public String statement() {
    double totalAmount = 0;
    int frequentRenterPoints = 0;
    Iterator<Rental> iterator = rentals.iterator();
    String result = "Rental Record for " + getName() + "\n";

    while ( iterator.hasNext() ) {
        Rental each = (Rental) iterator.next();
        // determine amounts for each line

        double thisAmount = amountFor(each);
    }
}
```

[실습] Legacy Code Refactoring

✓2. 자신이 사용하는 데이터와 같은 객체에 들어 있어야 한다.

- Move Instance Method (Rental class) & Rename Method

```
class Customer {  
    private double amountFor(Rental each) {  
        double thisAmount = 0;  
        switch (each.getMovie().getPriceCode()) {  
            case Movie.REGULAR:  
                thisAmount += 2;  
                if (each.getDaysRented() > 2)  
                    thisAmount += (each.getDaysRented() - 2) * 1.5;  
                break;  
  
            case Movie.NEW_RELEASE:  
                thisAmount += each.getDaysRented() * 3;  
                break;  
  
            case Movie.CHILDRENS:  
                thisAmount += 1.5;  
                if (each.getDaysRented() > 3)  
                    thisAmount += (each.getDaysRented() - 3) * 1.5;  
                break;  
        }  
        return thisAmount;  
    }  
}
```

Customer.java



```
class Rental {  
    public double getCharge() {  
        double thisAmount = 0;  
        switch (getMovie().getPriceCode()) {  
            case Movie.REGULAR:  
                thisAmount += 2;  
                if (getDaysRented() > 2)  
                    thisAmount += (getDaysRented() - 2) * 1.5;  
                break;  
  
            case Movie.NEW_RELEASE:  
                thisAmount += getDaysRented() * 3;  
                break;  
  
            case Movie.CHILDRENS:  
                thisAmount += 1.5;  
                if (getDaysRented() > 3)  
                    thisAmount += (getDaysRented() - 3) * 1.5;  
                break;  
        }  
        return thisAmount;  
    }  
}
```

Rental.java

[실습] Legacy Code Refactoring

✓3. Replace Temp with Query

- thisAmount inline

Customer.java

```
while ( iterator.hasNext() ) {  
    Rental each = (Rental) iterator.next();  
    // determine amounts for each line  
  
    double thisAmount = each.getCharge();  
  
    // add frequent renter points  
    frequentRenterPoints++;  
    // add bonus for a two day new release rental  
    if ((each.getMovie().getPriceCode() == Movie.NEW_RELEASE) && each.getDaysRented() > 1)  
        frequentRenterPoints++;  
    // show figures  
    result += "\t" + String.valueOf(thisAmount) + "(" + each.getMovie().getTitle() + ")" + "\n";  
  
    totalAmount += thisAmount;  
}
```



Customer.java

```
while ( iterator.hasNext() ) {  
    Rental each = (Rental) iterator.next();  
    // determine amounts for each line  
  
    // add frequent renter points  
    frequentRenterPoints++;  
    // add bonus for a two day new release rental  
    if ((each.getMovie().getPriceCode() == Movie.NEW_RELEASE) && each.getDaysRented() > 1)  
        frequentRenterPoints++;  
    // show figures  
    result += "\t" + String.valueOf(each.getCharge()) + "(" + each.getMovie().getTitle() + ")" + "\n";  
  
    totalAmount += each.getCharge();  
}
```

[실습] Legacy Code Refactoring

✓4. Extract Method – totalAmount

- Step 1. totalAmount 계산 구분을 while문 밖으로 꺼낸다.(새로운 for문 생성)
- Step 2. 변수(totalAmount)를 사용 직전에 선언하도록 한다.

```
double totalAmount = 0;
while ( iterator.hasNext() ) {
    Rental each = (Rental) iterator.next();
    // determine amounts for each line

    // add frequent renter points
    frequentRenterPoints++;
    // add bonus for a two day new release rental
    if ((each.getMovie().getPriceCode() == Movie.NEW_RELEASE) && each.getDaysRented() > 1)
        frequentRenterPoints++;
    // show figures
    result += "\t" + String.valueOf(each.getCharge()) + "(" + each.getMovie().getTitle() + ")" + "\n";

    totalAmount += each.getCharge();
}
```

Customer.java



```
while ( iterator.hasNext() ) {
    Rental each = (Rental) iterator.next();

    // add frequent renter points
    frequentRenterPoints++;
    // add bonus for a two day new release rental
    if ((each.getMovie().getPriceCode() == Movie.NEW_RELEASE) && each.getDaysRented() > 1)
        frequentRenterPoints++;
    // show figures
    result += "\t" + String.valueOf(each.getCharge()) + "(" + each.getMovie().getTitle() + ")" + "\n";
}

double totalAmount = 0;
for(Rental rental : rentals) {
    totalAmount += rental.getCharge();
}
```

Customer.java

[실습] Legacy Code Refactoring

✓4. Extract Method – totalAmount

- step 3. extract method

Customer.java

```
while ( iterator.hasNext() ) {  
    Rental each = (Rental) iterator.next();  
  
    // add frequent renter points  
    frequentRenterPoints++;  
    // add bonus for a two day new release rental  
    if ((each.getMovie().getPriceCode() == Movie.NEW_RELEASE) && each.getDaysRented() > 1)  
        frequentRenterPoints++;  
    // show figures  
    result += "\t" + String.valueOf(each.getCharge()) + "(" + each.getMovie().getTitle() + ")" + "\n";  
}  
  
double totalAmount = 0;  
for(Rental rental : rentals) {  
    totalAmount += rental.getCharge();  
}
```



Customer.java

```
private double getTotalAmount() {  
    double totalAmount = 0;  
    for(Rental rental : rentals) {  
        totalAmount += rental.getCharge();  
    }  
    return totalAmount;  
}
```

[실습] Legacy Code Refactoring

✓4. Extract Method – totalAmount

- Step 4. getTotalAmount() inline

Customer.java

```
while ( iterator.hasNext() ) {
    Rental each = (Rental) iterator.next();

    ...

    // show figures
    result += "\t" + String.valueOf(each.getCharge()) + "(" + each.getMovie().getTitle() + ")" + "\n";
}

double totalAmount = getTotalAmount();

result += "Amount owed is " + String.valueOf(totalAmount) + "\n";
result += "You earned " + String.valueOf(frequentRenterPoints) + " frequent renter pointers";
```



Customer.java

```
while ( iterator.hasNext() ) {
    Rental each = (Rental) iterator.next();

    ...

    // show figures
    result += "\t" + String.valueOf(each.getCharge()) + "(" + each.getMovie().getTitle() + ")" + "\n";
}

result += "Amount owed is " + getTotalAmount() + "\n";
result += "You earned " + String.valueOf(frequentRenterPoints) + " frequent renter pointers";
```

[실습] Legacy Code Refactoring

✓5. Extract Method - frequentRenterPoints

- Step 1. frequentRenterPoints 계산 구분을 while문 밖으로 꺼낸다.(새로운 for문 생성)
- Step 2. 변수(frequentRenterPoints)를 사용 직전에 선언하도록 한다.

```
int frequentRenterPoints = 0;
Iterator<Rental> iterator = rentals.iterator();
String result = "Rental Record for " + getName() + "\n";

while ( iterator.hasNext() ) {
    Rental each = (Rental) iterator.next();

    // add frequent renter points
    frequentRenterPoints++;
    // add bonus for a two day new release rental
    if ((each.getMovie().getPriceCode() == Movie.NEW_RELEASE) && each.getDaysRented() > 1)
        frequentRenterPoints++;
    // show figures
    result += "\t" + String.valueOf(each.getCharge())
}
}
```

Customer.java

Customer.java

```
Iterator<Rental> iterator = rentals.iterator();
String result = "Rental Record for " + getName() + "\n";

while ( iterator.hasNext() ) {
    Rental each = (Rental) iterator.next();

    // show figures
    result += "\t" + String.valueOf(each.getCharge()) + "(" + each.getMovie().getTitle() + ")" + "\n";
}

int frequentRenterPoints = 0;
for(Rental rental : rentals) {
    frequentRenterPoints++;
    // add bonus for a two day new release rental
    if ((rental.getMovie().getPriceCode() == Movie.NEW_RELEASE) && rental.getDaysRented() > 1)
        frequentRenterPoints++;
}
}
```

[실습] Legacy Code Refactoring

✓5. Extract Method - frequentRenterPoints

- step 3. extract method

```
Iterator<Rental> iterator = rentals.iterator();
String result = "Rental Record for " + getName() + "\n";

while ( iterator.hasNext() ) {
    Rental each = (Rental) iterator.next();

    // show figures
    result += "\t" + String.valueOf(each.getCharge()) + "(" + each.getMovie().getTitle() + ")" + "\n";
}

int frequentRenterPoints = 0;
for(Rental rental : rentals) {           // add frequent renter points
    frequentRenterPoints++;
    // add bonus for a two day new release rental
    if ((rental.getMovie().getPriceCode() == Movie.NEW_RELEASE) && rental.getDaysRented() > 1)
        frequentRenterPoints++;
}
```

Customer.java



```
private int getFrequentRenterPoints() {
    int frequentRenterPoints = 0;
    for(Rental rental : rentals) {           // add frequent renter points
        frequentRenterPoints++;
        // add bonus for a two day new release rental
        if ((rental.getMovie().getPriceCode() == Movie.NEW_RELEASE) && rental.getDaysRented() > 1)
            frequentRenterPoints++;
    }
    return frequentRenterPoints;
}
```

Customer.java

[실습] Legacy Code Refactoring

✓5. Extract Method - frequentRenterPoints

- Step 4. getFrequentRenterPoints() inline

Customer.java

```
public String statement() {
    Iterator<Rental> iterator = rentals.iterator();
    String result = "Rental Record for " + getName() + "\n";

    while ( iterator.hasNext() ) {
        ...
    }

    int frequentRenterPoints = getFrequentRenterPoints();

    result += "Amount owed is " + getTotalAmount() + "\n";
    result += "You earned " + String.valueOf(frequentRenterPoints) + " frequent renter pointers";

    return result;
}
```

Customer.java



```
public String statement() {
    Iterator<Rental> iterator = rentals.iterator();
    String result = "Rental Record for " + getName() + "\n";

    while ( iterator.hasNext() ) {
        Rental each = (Rental) iterator.next();

        // show figures
        result += "\t" + String.valueOf(each.getCharge()) + "(" + each.getMovie().getTitle() + ")" + "\n";
    }

    result += "Amount owed is " + getTotalAmount() + "\n";
    result += "You earned " + getFrequentRenterPoints() + " frequent renter pointers";

    return result;
}
```

[실습] Legacy Code Refactoring

✓ 5. Extract Method - frequentRenterPoints

- Step 5. frequentRenterPoints 합산하는 부분과 증가 로직 부분을 분리시켜 메서드로 뺀다.

Customer.java

```
private int getFrequentRenterPoints() {  
    int frequentRenterPoints = 0;  
    for(Rental rental : rentals) {           // add frequent renter points  
        frequentRenterPoints++;  
        // add bonus for a two day new release rental  
        if ((rental.getMovie().getPriceCode() == Movie.NEW_RELEASE) && rental.getDaysRented() > 1)  
            frequentRenterPoints++;  
    }  
    return frequentRenterPoints;  
}
```



Customer.java

```
private int getFrequentRenterPoints() {  
    int frequentRenterPoints = 0;  
    for(Rental rental : rentals) {           // add frequent renter points  
        frequentRenterPoints = getFrequentRenterPointsFor(frequentRenterPoints, rental);  
    }  
    return frequentRenterPoints;  
}  
  
private int getFrequentRenterPointsFor(int frequentRenterPoints, Rental rental) {  
    frequentRenterPoints++;  
    // add bonus for a two day new release rental  
    if ((rental.getMovie().getPriceCode() == Movie.NEW_RELEASE) && rental.getDaysRented() > 1)  
        frequentRenterPoints++;  
    return frequentRenterPoints;  
}
```


[실습] Legacy Code Refactoring

✓5. Extract Method - frequentRenterPoints

- Step 6. getFrequentRenterPointsFor에서 frequentRenterPoints 파라미터 삭제

Customer.java

```
private int getFrequentRenterPoints() {
    int frequentRenterPoints = 0;
    for(Rental rental : rentals) {           // add frequent renter points
        frequentRenterPoints = getFrequentRenterPointsFor(frequentRenterPoints, rental);
    }
    return frequentRenterPoints;
}

private int getFrequentRenterPointsFor(int frequentRenterPoints, Rental rental) {
    frequentRenterPoints++;
    // add bonus for a two day new release rental
    if ((rental.getMovie().getPriceCode() == Movie.NEW_RELEASE) && rental.getDaysRented() > 1)
        frequentRenterPoints++;
    return frequentRenterPoints;
}
```



Customer.java

```
private int getFrequentRenterPoints() {
    int frequentRenterPoints = 0;
    for(Rental rental : rentals) {           // add frequent renter points
        frequentRenterPoints += getFrequentRenterPointsFor(rental);
    }
    return frequentRenterPoints;
}

private int getFrequentRenterPointsFor(Rental rental) {
    int frequentRenterPoints = 0;
    frequentRenterPoints++;
    // add bonus for a two day new release rental
    if ((rental.getMovie().getPriceCode() == Movie.NEW_RELEASE) && rental.getDaysRented() > 1)
        frequentRenterPoints++;
    return frequentRenterPoints;
}
```

[실습] Legacy Code Refactoring

✓5. Extract Method - frequentRenterPoints

- Step 7. getFrequentRenterPointsFor 로직을 더 간단하게 수정할 수 있다.

Customer.java

```
private int getFrequentRenterPointsFor(Rental rental) {  
    int frequentRenterPoints = 0;  
    frequentRenterPoints++;  
    // add bonus for a two day new release rental  
    if ((rental.getMovie().getPriceCode() == Movie.NEW_RELEASE) && rental.getDaysRented() > 1)  
        frequentRenterPoints++;  
    return frequentRenterPoints;  
}
```



Customer.java

```
private int getFrequentRenterPointsFor(Rental rental) {  
    // add bonus for a two day new release rental  
    if ((rental.getMovie().getPriceCode() == Movie.NEW_RELEASE) && rental.getDaysRented() > 1) {  
        return 2;  
    } else {  
        return 1;  
    }  
}
```

[실습] Legacy Code Refactoring

✓6. Move Method (Rental class)

- 자신이 사용하는 데이터와 같은 객체에 들어 있어야 한다.
- Customer class 에서 Rental class 로 이동

```
private int getFrequentRenterPointsFor(Rental rental) {  
    // add bonus for a two day new release rental  
    if ((rental.getMovie().getPriceCode() == Movie.NEW_RELEASE) && rental.getDaysRented() > 1) {  
        return 2;  
    } else {  
        return 1;  
    }  
}
```

Customer.java



```
class Rental {  
    int getFrequentRenterPointsFor() {  
        // add bonus for a two day new release rental  
        if ((getMovie().getPriceCode() == Movie.NEW_RELEASE) && getDaysRented() > 1) {  
            return 2;  
        } else {  
            return 1;  
        }  
    }  
}
```

Rental.java

[실습] Legacy Code Refactoring

✓7. Print Logic Composed Method

- Statement Method의 가독성을 위해 Print Logic을 Method로 구성한다.
- Step 1. 변수(iterator)를 사용 직전에 선언하도록 한다.

```
public String statement() {  
    Iterator<Rental> iterator = rentals.iterator();  
    String result = "Rental Record for " + getName() + "\n";  
  
    while ( iterator.hasNext() ) {  
        Rental each = (Rental) iterator.next();  
        result += "\t" + each.getCharge() + "(" + each.getMovie().getTitle() + ")" + "\n";  
    }  
  
    result += "Amount owed is " + getTotalAmount() + "\n";  
    result += "You earned " + getFrequentRenterPoints() + " frequent renter pointers";  
  
    return result;  
}
```

Customer.java

```
public String statement() {  
    String result = "Rental Record for " + getName() + "\n";  
    Iterator<Rental> iterator = rentals.iterator();  
    while ( iterator.hasNext() ) {  
        Rental each = (Rental) iterator.next();  
        result += "\t" + each.getCharge() + "(" + each.getMovie().getTitle() + ")" + "\n";  
    }  
  
    result += "Amount owed is " + getTotalAmount() + "\n";  
    result += "You earned " + getFrequentRenterPoints() + " frequent renter pointers";  
  
    return result;  
}
```

Customer.java

[실습] Legacy Code Refactoring

✓7. Print Logic Composed Method

■ Step 2. Extract method

```
public String statement() {  
    String result = "Rental Record for " + getName() + "\n";  
  
    Iterator<Rental> iterator = rentals.iterator();  
    while ( iterator.hasNext() ) {  
        Rental each = (Rental) iterator.next();  
        result += "\t" + each.getCharge() + "(" + each.getMovie().getTitle() + ")" + "\n";  
    }  
  
    result += "Amount owed is " + getTotalAmount() + "\n";  
    result += "You earned " + getFrequentRenterPoints() + " frequent renter pointers";  
  
    return result;  
}
```

Customer.java

```
public String statement() {  
    String result = "Rental Record for " + getName() + "\n";  
    result = getRentalLineReport(result);  
    result += "Amount owed is " + getTotalAmount() + "\n";  
    result += "You earned " + getFrequentRenterPoints() + " frequent renter pointers";  
  
    return result;  
}  
  
private String getRentalLineReport(String result) {  
    Iterator<Rental> iterator = rentals.iterator();  
    while ( iterator.hasNext() ) {  
        Rental each = (Rental) iterator.next();  
        result += "\t" + each.getCharge() + "(" + each.getMovie().getTitle() + ")" + "\n";  
    }  
    return result;  
}
```

Customer.java



[실습] Legacy Code Refactoring

✓7. Print Logic Composed Method

- Step 3. getRentalLineReport에서 파라미터/Iterator 삭제

```
public String statement() {
    String result = "Rental Record for " + getName() + "\n";
    result = getRentalLineReport(result);
    result += "Amount owed is " + getTotalAmount() + "\n";
    result += "You earned " + getFrequentRenterPoints() + " frequent renter pointers";

    return result;
}

private String getRentalLineReport(String result) {
    Iterator<Rental> iterator = rentals.iterator();
    while ( iterator.hasNext() ) {
        Rental each = (Rental) iterator.next();
        result += "\t" + each.getCharge() + "(" + each.getMovie().getTitle() + ")" + "\n";
    }
    return result;
}
```

Customer.java

```
public String statement() {
    String result = "Rental Record for " + getName() + "\n";
    result += getRentalLineReport();
    result += "Amount owed is " + getTotalAmount() + "\n";
    result += "You earned " + getFrequentRenterPoints() + " frequent renter pointers";

    return result;
}

private String getRentalLineReport() {
    String result = "";
    for ( Rental rental : rentals ) {
        result += "\t" + rental.getCharge() + "(" + rental.getMovie().getTitle() + ")" + "\n";
    }
    return result;
}
```

Customer.java

[실습] Legacy Code Refactoring

✓7. Print Logic Composed Method

- Step 4. 나머지 Print Logic도 각각의 Method로 구성한다.

```
Customer.java
public String statement() {
    String result = "Rental Record for " + getName() + "\n";
    result += getRentalLineReport();
    result += "Amount owed is " + getTotalAmount() + "\n";
    result += "You earned " + getFrequentRenterPoints() + " frequent renter pointers";

    return result;
}
```



```
Customer.java
public String statement() {
    String result = getStatementHeader();
    result += getRentalLineReport();
    result += getStatementFooter();

    return result;
}

private String getStatementHeader() {
    return "Rental Record for " + getName() + "\n";
}

private String getStatementFooter() {
    String result = "Amount owed is " + getTotalAmount() + "\n";
    result += "You earned " + getFrequentRenterPoints() + " frequent renter pointers";
    return result;
}
```

[실습] Legacy Code Refactoring

✓8. Move Method (Rental class -> Movie class) – getCharge()

- Switch 문의 인자로 자신의 객체 데이터를 사용해야 한다.
- 또한, Movie 종류가 추가되면 Rental 객체를 수정해야 하므로 Movie 로 옮겨야 한다.
- Step 1. extract variable (daysRented)

Rental.java

```
public double getCharge() {
    double thisAmount = 0;
    switch (getMovie().getPriceCode()) {
        case Movie.REGULAR:
            thisAmount += 2;
            if (getDaysRented() > 2)
                thisAmount += (getDaysRented() - 2) * 1.5;
            break;

        case Movie.NEW_RELEASE:
            thisAmount += getDaysRented() * 3;
            break;

        case Movie.CHILDRENS:
            thisAmount += 1.5;
            if (getDaysRented() > 3)
                thisAmount += (getDaysRented() - 3) * 1.5;
            break;
    }
    return thisAmount;
}
```



Rental.java

```
public double getCharge() {
    int daysRented = getDaysRented();

    double thisAmount = 0;
    switch (getMovie().getPriceCode()) {
        case Movie.REGULAR:
            thisAmount += 2;
            if (daysRented > 2)
                thisAmount += (daysRented - 2) * 1.5;
            break;

        case Movie.NEW_RELEASE:
            thisAmount += daysRented * 3;
            break;

        case Movie.CHILDRENS:
            thisAmount += 1.5;
            if (daysRented > 3)
                thisAmount += (daysRented - 3) * 1.5;
            break;
    }
    return thisAmount;
}
```


[실습] Legacy Code Refactoring

✓ 8. Move Method (Rental class -> Movie class) – getCharge()

- Step 2. extract method (getChargeFor)

Rental.java

```
public double getCharge() {
    int daysRented = getDaysRented();

    double thisAmount = 0;
    switch (getMovie().getPriceCode()) {
        case Movie.REGULAR:
            thisAmount += 2;
            if (daysRented > 2)
                thisAmount += (daysRented - 2) * 1.5;
            break;

        case Movie.NEW_RELEASE:
            thisAmount += daysRented * 3;
            break;

        case Movie.CHILDRENS:
            thisAmount += 1.5;
            if (daysRented > 3)
                thisAmount += (daysRented - 3) * 1.5;
            break;
    }
    return thisAmount;
}
```



Rental.java

```
public double getCharge() {
    return getChargeFor(getDaysRented());
}

private double getChargeFor(int daysRented) {
    double thisAmount = 0;
    switch (getMovie().getPriceCode()) {
        case Movie.REGULAR:
            thisAmount += 2;
            if (daysRented > 2)
                thisAmount += (daysRented - 2) * 1.5;
            break;

        case Movie.NEW_RELEASE:
            thisAmount += daysRented * 3;
            break;

        case Movie.CHILDRENS:
            thisAmount += 1.5;
            if (daysRented > 3)
                thisAmount += (daysRented - 3) * 1.5;
            break;
    }
    return thisAmount;
}
```

[실습] Legacy Code Refactoring

✓ 8. Move Method (Rental class -> Movie class) – getCharge()

- Step 3. move getChargeFor method (Rental class -> Movie class)

Rental.java

```
public double getCharge() {
    return getChargeFor(getDaysRented());
}

private double getChargeFor(int daysRented) {
    double thisAmount = 0;
    switch (getMovie().getPriceCode()) {
        case Movie.REGULAR:
            thisAmount += 2;
            if (daysRented > 2)
                thisAmount += (daysRented - 2) * 1.5;
            break;

        case Movie.NEW_RELEASE:
            thisAmount += daysRented * 3;
            break;

        case Movie.CHILDRENS:
            thisAmount += 1.5;
            if (daysRented > 3)
                thisAmount += (daysRented - 3) * 1.5;
            break;
    }
    return thisAmount;
}
```



Movie.java

```
double getChargeFor(int daysRented, Rental rental) {
    double thisAmount = 0;
    switch (rental.getMovie().getPriceCode()) {
        case REGULAR:
            thisAmount += 2;
            if (daysRented > 2)
                thisAmount += (daysRented - 2) * 1.5;
            break;

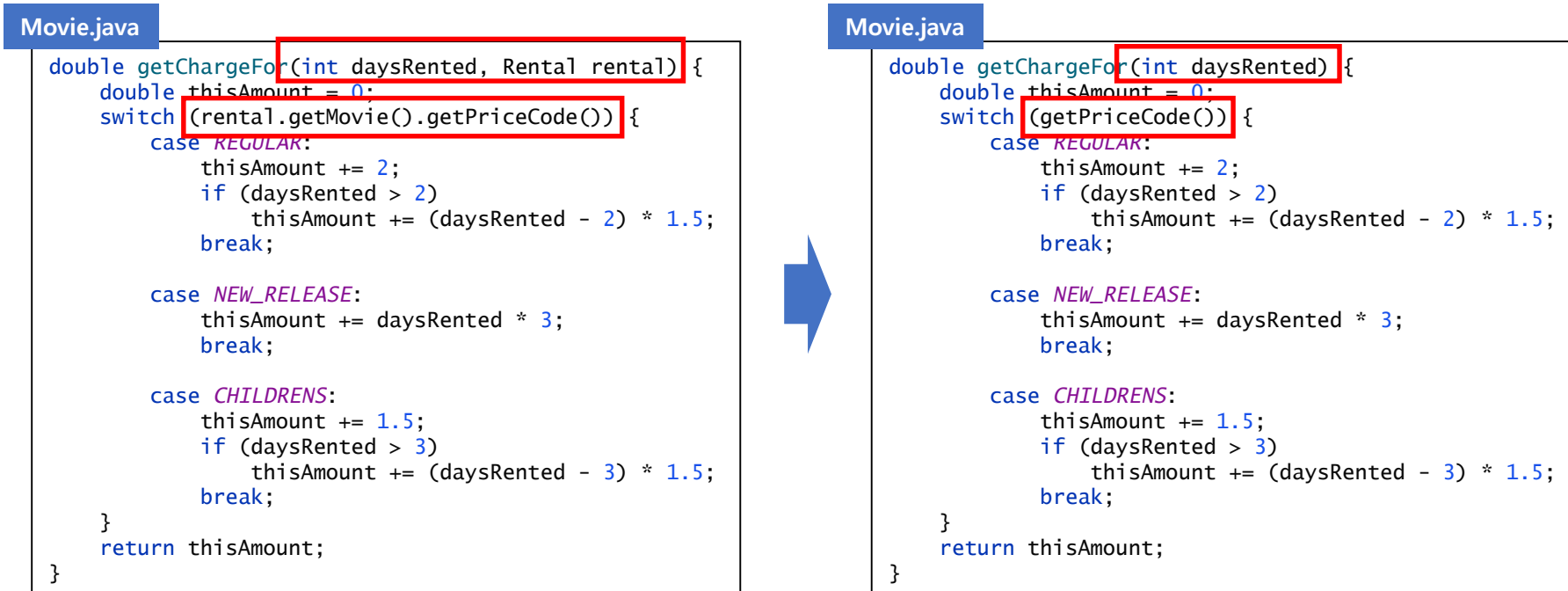
        case NEW_RELEASE:
            thisAmount += daysRented * 3;
            break;

        case CHILDRENS:
            thisAmount += 1.5;
            if (daysRented > 3)
                thisAmount += (daysRented - 3) * 1.5;
            break;
    }
    return thisAmount;
}
```

[실습] Legacy Code Refactoring

✓ 8. Move Method (Rental class -> Movie class) – getCharge()

- Step 4. rental 파라미터 삭제



[실습] Legacy Code Refactoring

✓ 8. Move Method (Rental class -> Movie class) – getFrequentRenterPointsFor()

- 적립 포인트 계산은 비디오 분류가 든 클래스 자체에서 처리해야 한다.
- getCharge() 와 같이 getFrequentRenterPointsFor()를 Movie class로 이동시킨다.

Rental.java

```
int getFrequentRenterPointsFor() {  
    // add bonus for a two day new release rental  
    if ((getMovie().getPriceCode() == Movie.NEW_RELEASE) && getDaysRented() > 1) {  
        return 2;  
    }else {  
        return 1;  
    }  
}
```



Movie.java

```
int getFrequentRenterPointsFor(int daysRented) {  
    if ((getPriceCode() == NEW_RELEASE) && daysRented > 1) {  
        return 2;  
    }else {  
        return 1;  
    }  
}
```

[실습] Legacy Code Refactoring

✓9. Replace Type code with Subclass

- 종류에 따른 호출시 각기 다른 값을 반환하기 위해 상속받는 하위클래스를 작성한다.

Movie.java

```
double getChargeFor(int daysRented) {  
    double thisAmount = 0;  
    switch (getPriceCode()) {  
        case REGULAR:  
            thisAmount += 2;  
            if (daysRented > 2)  
                thisAmount += (daysRented - 2) * 1.5;  
            break;  
  
        case NEW_RELEASE:  
            thisAmount += daysRented * 3;  
            break;  
  
        case CHILDRENS:  
            thisAmount += 1.5;  
            if (daysRented > 3)  
                thisAmount += (daysRented - 3) * 1.5;  
            break;  
    }  
    return thisAmount;  
}
```

[실습] Legacy Code Refactoring

✓9. Replace Type code with Subclass

- Step 1. subclass RegularMovie 를 생성한다.

Alt + Enter

Movie.java

```
public class Movie {  
    public stat  
    public stat  
    public stat  
    private Str  
    private int
```

Make 'Movie' package-private
Create Test
Create subclass
Add Javadoc
Press Ctrl+Shift+I to open preview

RegularMovie.java

```
package com.videorental;  
  
public class RegularMovie extends Movie {  
    public RegularMovie(String title) {  
        super(title, Movie.REGULAR);  
    }  
  
    @Override  
    double getChargeFor(int daysRented) {  
        double thisAmount = 2;  
        if (daysRented > 2)  
            thisAmount += (daysRented - 2) * 1.5;  
  
        return thisAmount;  
    }  
}
```

[실습] Legacy Code Refactoring

✓9. Replace Type code with Subclass - Test Code 수정

- Step 2. 테스트 코드 중 Movie 생성 부분을 메소드로 빼낸다. (extract method)
- Step 3. 영화 종류에 따라서 다른 Movie 객체를 리턴하도록 수정한다.

CustomerTest.java

```
private Rental createRentalFor(int daysRented, int priceCode) {  
    Movie movie = new Movie(TITLE, priceCode);  
    return new Rental(movie, daysRented);  
}
```

CustomerTest.java

```
private Rental createRentalFor(int daysRented, int priceCode) {  
    Movie movie = getMovie(priceCode);  
    return new Rental(movie, daysRented);  
}
```

```
private Movie getMovie(int priceCode) {  
    Movie movie = new Movie(TITLE, priceCode);  
    return movie;  
}
```

CustomerTest.java

```
private Movie getMovie(int priceCode) {  
    switch (priceCode){  
        case Movie.REGULAR:  
            return new RegularMovie(TITLE);  
        default :  
            return new Movie(TITLE, priceCode);  
    }  
}
```

[실습] Legacy Code Refactoring

✓9. Replace Type code with Subclass - NewReleaseMovie, ChildrenMovie

- Step 4. 동일한 방법으로 NewReleaseMovie, Children Movie Class를 생성하고, Test 코드를 수정한다.

NewReleaseMovie.java

```
package com.videorental;

public class NewReleaseMovie extends Movie {
    public NewReleaseMovie(String title) {
        super(title, Movie.NEW_RELEASE);
    }

    @Override
    double getChargeFor(int daysRented) {
        double thisAmount = 0;
        thisAmount += daysRented * 3;

        return thisAmount;
    }
}
```

ChildrenMovie.java

```
package com.videorental;

public class ChildrenMovie extends Movie {
    public ChildrenMovie(String title) {
        super(title, Movie.CHILDRENS);
    }

    @Override
    double getChargeFor(int daysRented) {
        double thisAmount = 0;
        thisAmount += 1.5;
        if (daysRented > 3)
            thisAmount += (daysRented - 3) * 1.5;
        return thisAmount;
    }
}
```

CustomerTest.java

```
private Movie getMovie(int priceCode) {
    switch (priceCode){
        case Movie.REGULAR:
            return new RegularMovie(TITLE);
        case Movie.NEW_RELEASE:
            return new NewReleaseMovie(TITLE);
        case Movie.CHILDRENS:
            return new ChildrenMovie(TITLE);
        default :
            return null;
    }
}
```


[실습] Legacy Code Refactoring

✓9. Replace Type code with Subclass – abstract class

- Step 5. 모두 변경 후 Movie 객체와 getChargeFor 메서드를 abstract 메서드로 변경한다.

Movie.java

```
public abstract class Movie {  
    abstract double getChargeFor(int daysRented);  
}
```

[실습] Legacy Code Refactoring

✓9. Replace Type code with Subclass

- Step 6. frequentRenterPointsFor 에 대해서도 동일한 방식으로 변경한다.
- NewReleaseMovie 에 대해서만 로직이 들어가고 나머지 클래스에서는 1을 반환한다.
- 모두 변경 후 Movie 의 frequentRenterPointsFor 메서드는 abstract 메서드로 변경한다.

ChildrenMovie.java

```
@Override
int getFrequentRenterPointsFor(int daysRented) {
    return 1;
}
```

RegularMovie.java

Movie.java

```
public abstract class Movie {

    abstract double getChargeFor(int daysRented);

    abstract int getFrequentRenterPointsFor(int daysRented);

}
```

NewReleaseMovie.java

```
@Override
int getFrequentRenterPointsFor(int daysRented) {
    if(daysRented > 1) {
        return 2;
    }else {
        return 1;
    }
}
```

감사합니다.